

GOTTA BREAK EM' ALL

# SMASHROID

START GAME



## GAME OVERVIEW

# ACTION ADVENTURE

This game is a simple console-based space shooter where the player controls a spacecraft placed at the bottom of the screen and must survive incoming asteroids falling from the top. The screen is divided into five lanes, and the player can move left or right to avoid collisions via A and D respectively.

The player can also shoot bullets to destroy asteroids before they reach the bottom with the spacebar. Different sizes of asteroids appear, and each type gives different points when destroyed. The main objective of the game is to score as high as possible by destroying asteroids while preventing them from hitting the spacecraft.

**NEXT SLIDE**





# YOUR MISSION

The player's mission is to control a spacecraft and survive by avoiding asteroids falling from the top of the screen. The player can move left or right between five lanes to prevent collisions.

01

The player must also shoot bullets to destroy asteroids and earn points. The main goal is to destroy as many asteroids as possible and achieve the highest score.

02

NEXT SLIDE



# KNOW YOUR ROCKS

SMALL

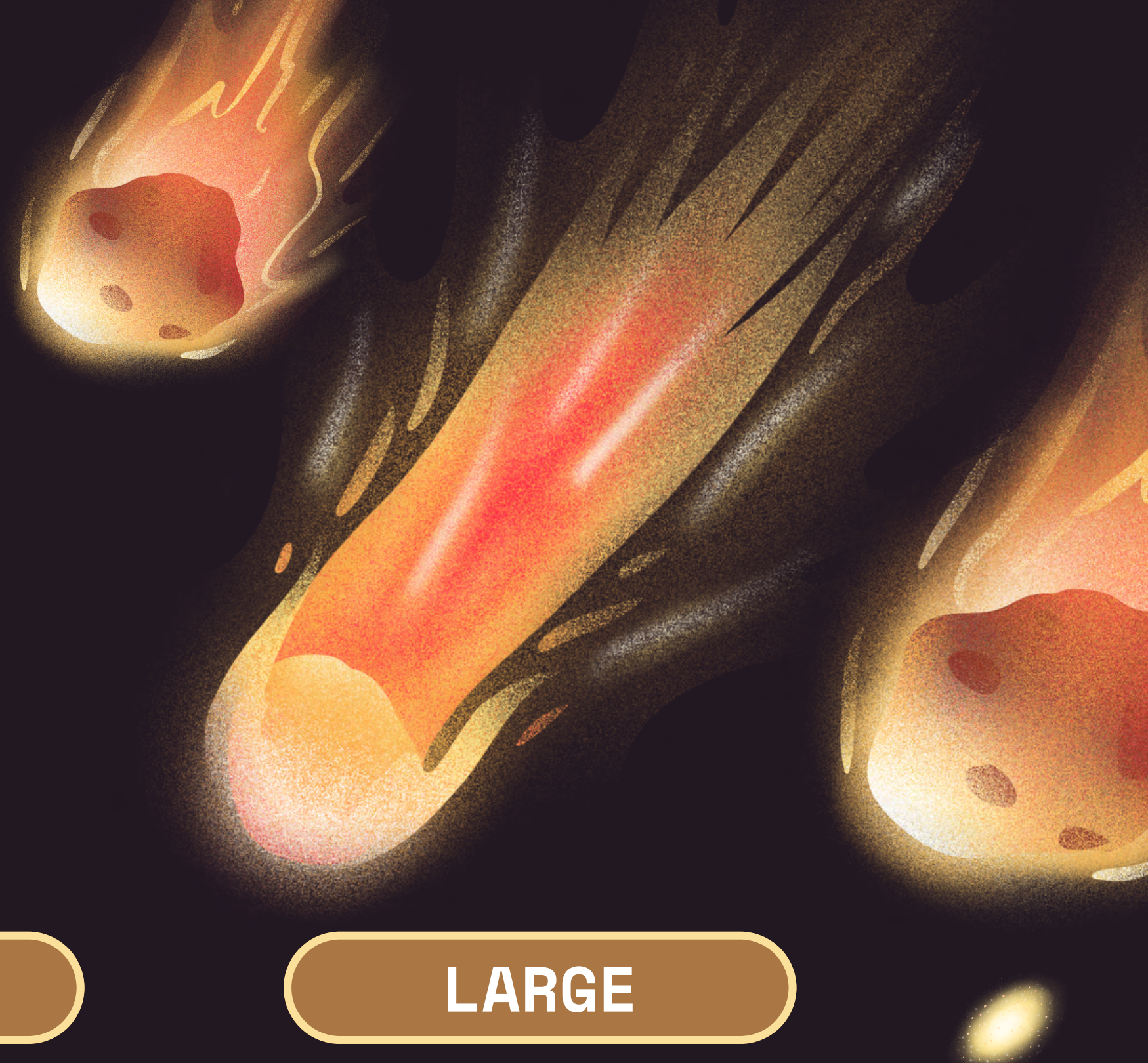
HEIGHT: 1 ROW  
WORTH: 1 POINT  
SPEED: FAST

MEDIUM

HEIGHT: 2 ROWS  
WORTH: 3 POINTS  
SPEED: NORMAL

LARGE

HEIGHT: 3 ROWS  
WORTH: 5 POINTS  
SPEED: SLOW





## EASY CONSOLE-BASED GRAPHICS

C allows printing and updating the screen quickly using `printf()` and `system("cls")`

## EFFICIENT INPUT HANDLING:

Using `conio.h` functions like `kbhit()` and `getch()` gives instant keyboard response for movement/shooting.

## FAST EXECUTION

Game loop runs smoothly without lag while asteroids move and bullets shoot

## LOW MEMORY USAGE

Arrays for asteroids and bullets take very little memory, making the game lightweight.

## BETTER OPTIMIZATION

One can control game speed using `Sleep()` which makes gameplay adjustable.

# WHY C?



# LIBRARIES

## <TIME.H>

Used for:

- `time (0)` to get the current time
- helps in random asteroid generation using:  
`srand(time(0))`

## <WINDOWS.H>

Used for Windows-specific functions:

- `Sleep()` (delay / control speed)

## <CONIO.H>

Used for keyboard input without Enter, like in games:

- `kbhit()` (checks if key pressed)
- `getch()` (takes key input instantly)

## <STDLIB.H>

- `rand()` (random numbers)
- `srand()` (random seed)

## <STDIO.H>

Used for input/output functions like:

- `printf()`



## ASTEROID

- `asteroidHeight()`: returns the asteroid size
- `asteroidPoints()`: returns points based on asteroid type

## TIME

- `srand(time(0))` is used for generating random seeds
- `time(0)` is used to get the current time

## INPUT

- `kbhit()` is used to check if a key is pressed
- `getch()` reads the pressed key instantly

# FUNCTIONS



# FUNCTIONS

## SYSTEM

- `Main()` is used to run the entire game
- `printf()` is used to print the elements of the game like asteroids and the spaceship

## RESET

`system("cls")` is used to clear the screen and reset the game, this can be used for instances where there is a new user.



# CONCLUSION

01

The Asteroids Game project helped us understand the practical use of C programming concepts in game development. Through this project, we applied concepts like loops, functions, structures, conditional statements, and keyboard controls.

02

We learned how to:

- Create moving objects on the screen
- Detect collisions between the spaceship and asteroids
- Use game logic and scoring systems
- Improve problem-solving and logical thinking

03

This project also showed how basic programming knowledge can be used to build interactive and fun applications. Overall, the Asteroids game strengthened our understanding of C language and improved our coding skills in a creative way.



The background is a dark, deep blue space. In the top left, there is a cluster of grey, cratered asteroids of various sizes. In the top right, a large, textured reddish-brown planet is partially visible. In the bottom left, a large portion of a planet with horizontal orange and white stripes is shown. In the bottom right, another cluster of grey, cratered asteroids is present. Two bright, glowing yellow galaxies or nebulae are scattered in the lower half of the image, one near the bottom center and one near the bottom right.

**CHALLENGE  
YOUR  
FRIENDS!**